Programming Language 1
Python

Dr. Samera Obaid Barraood

FOR LOOP IN PYTHON AND USING LISTS

AGENDA

- FOR LOOP IN PYTHON
- INDENTATION IN PYTHON
- LIST COMPREHENSIONS
- SLICING A LIST
- COPYING A LIST

PRINT EACH NAME IN THE LIST

```
names = ['Shivam', 'Navneet', 'Anil']

print(names[0])

print(names[1])

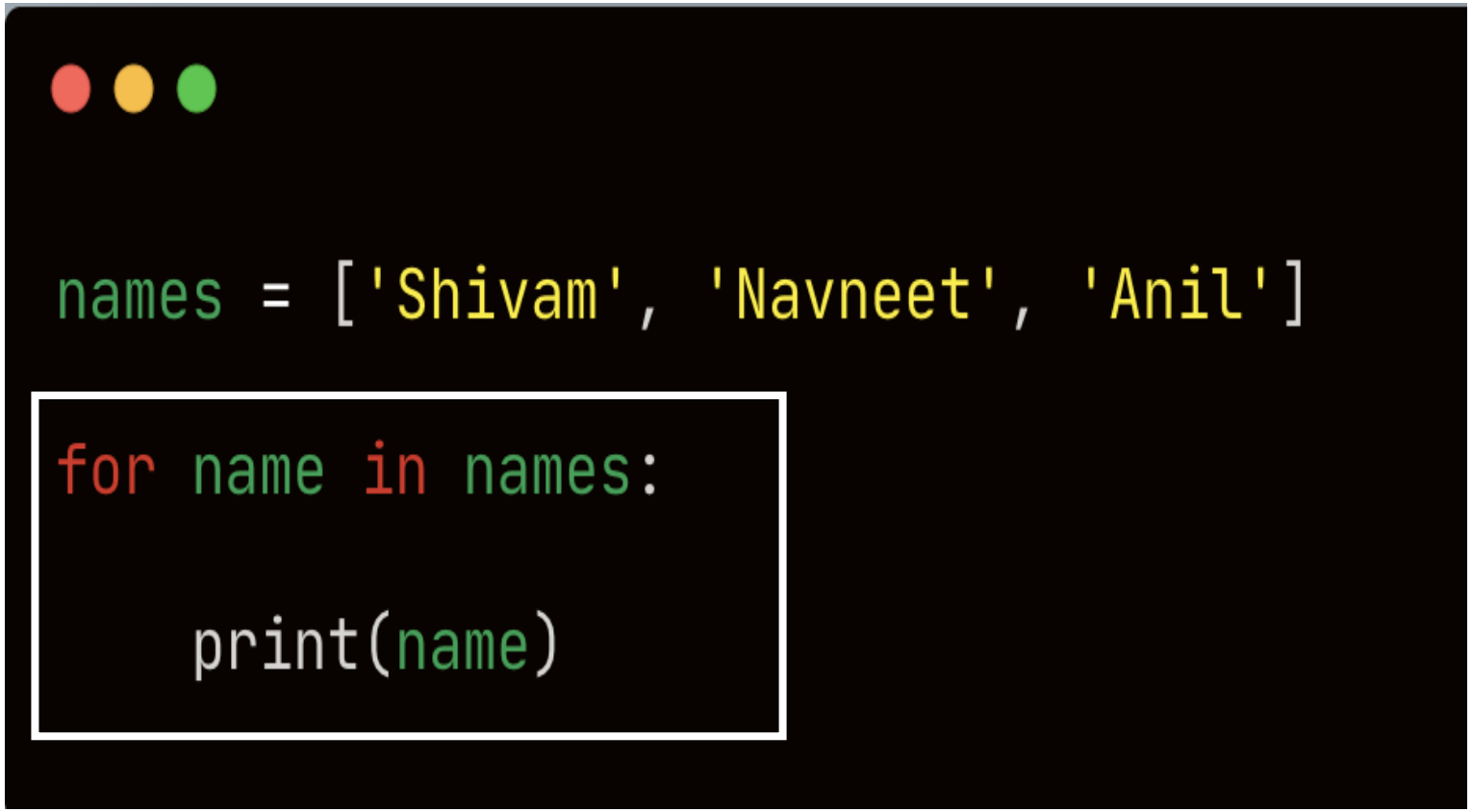
print(names[2])

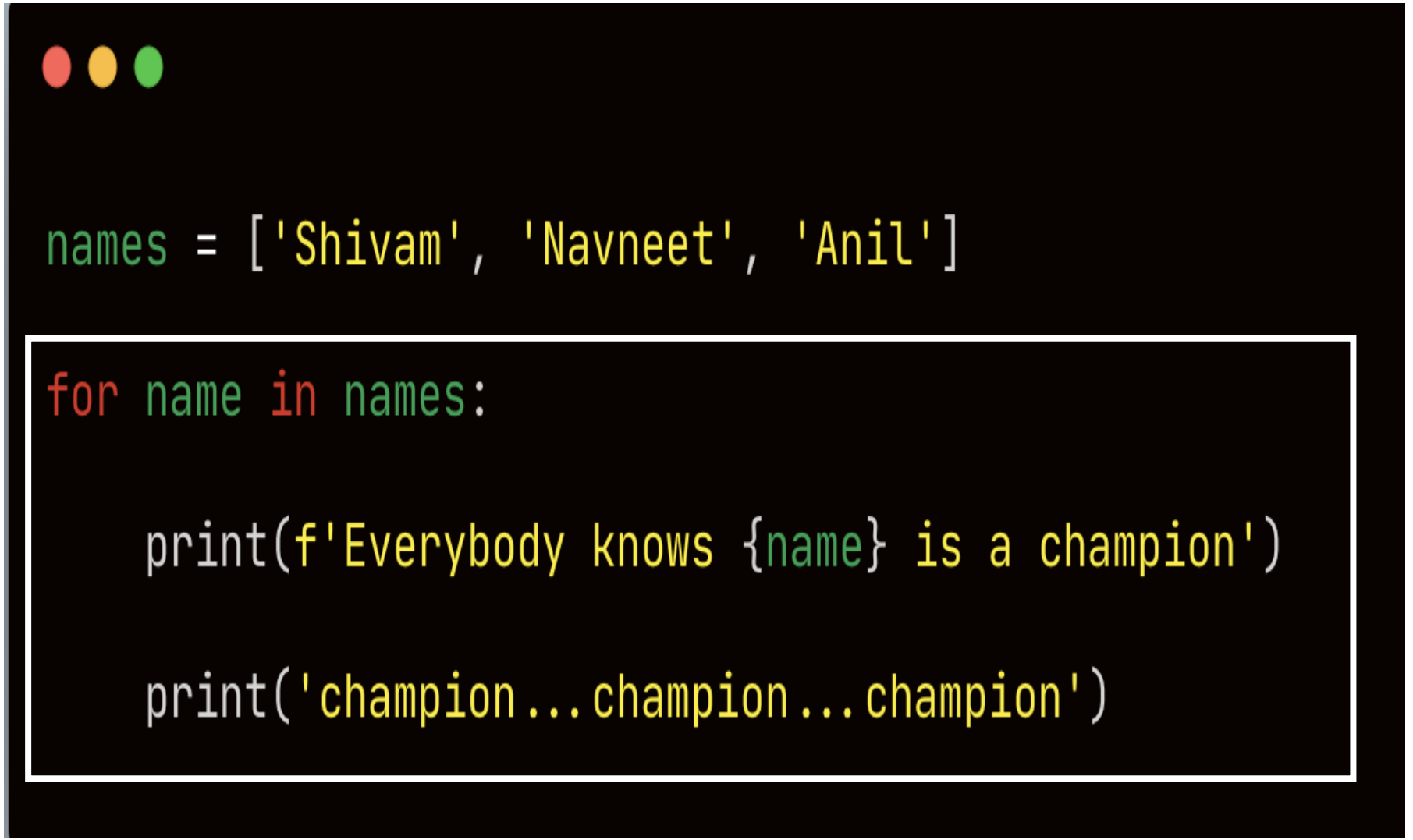
print(names[0], names[1], names[2])
```

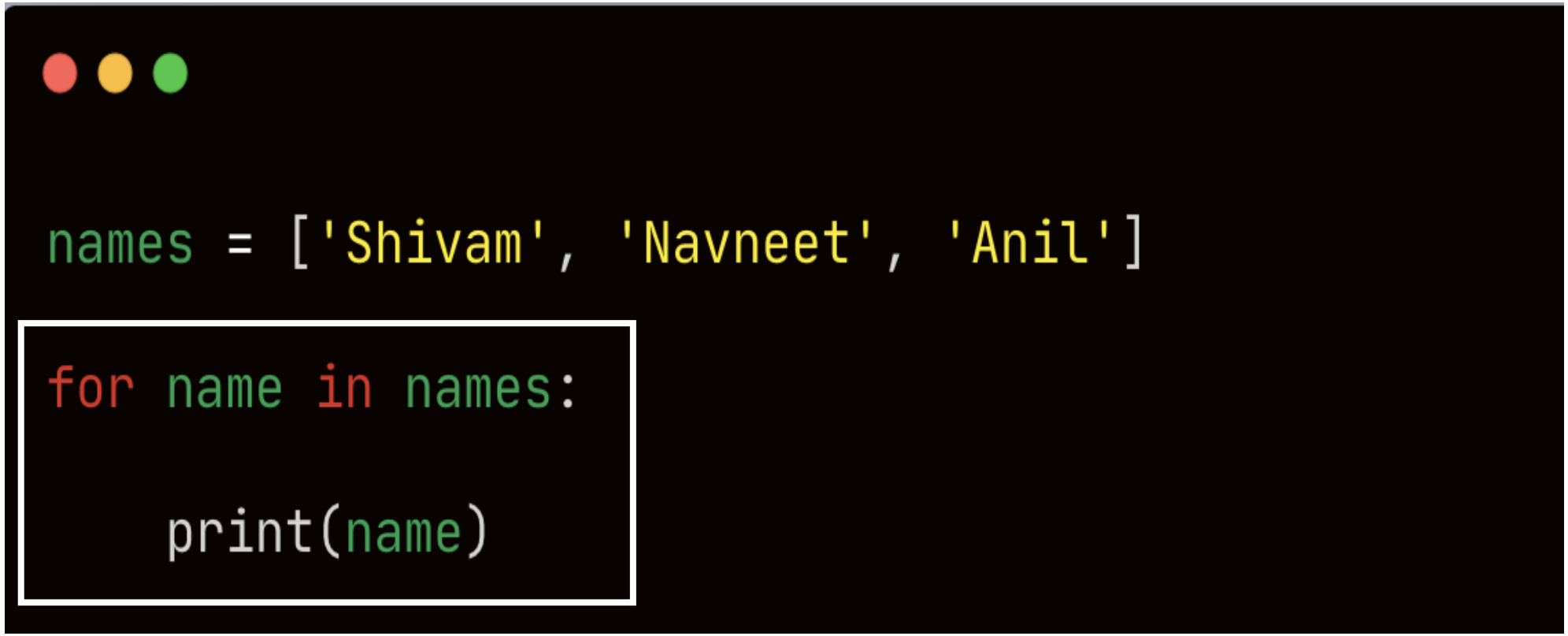
ISSUES WITH THIS APPROACH

- Repetitive code
- Need to change the code if the list size changes

FOR LOOP FOR PRINTING THE LIST




names = ['Shivam', 'Navneet', 'Anil']
for name in names:
print(f'Everybody knows {name} is a champion')
print('champion...champion...champion')


names = ['Shivam', 'Navneet', 'Anil']
for name in names:
print(name)

EXPLAINING HOW FOR LOOP WORKED

When you're using loops for the first time, keep in mind that the set of steps is repeated once for each item in the list, no matter how many items are in the list. If you have a million items in your list, Python repeats these steps a million times—and usually very quickly.

Also keep in mind when writing your own `for` loops that you can choose any name you want for the temporary variable that will be associated with each value in the list. However, it's helpful to choose a meaningful name that represents a single item from the list. For example, here's a good way to start a `for` loop for a list of cats, a list of dogs, and a general list of items:

```
for cat in cats:
for dog in dogs:
for item in list_of_items:
```

These naming conventions can help you follow the action being done on each item within a `for` loop. Using singular and plural names can help you identify whether a section of code is working with a single element from the list or the entire list.

AVOIDING INDENTATION ERRORS

WHAT IS INDENTATION

- Indentation is used to make code more readability easier
- In languages like c, c++, c# etc. , indentation is optional
- In Python, it is mandatory
- Python uses indentation to determine how a line, or group of lines, is related to the rest of the program.
- Change indentation and the meaning of code changes

INDENTATION

```c
int numbers[] = {1, 2, 3};

for(int i = 0; i < 3; i++) {

    printf("%d\n",numbers[i]);

}
```

```c
int numbers[] = {1, 2, 3};

for(int i = 0; i < 3; i++)

{

    printf("%d\n",numbers[i]);

}
```

As you begin to write code that relies on proper indentation, you'll need to watch for a few common *indentation errors*. For example, people sometimes indent lines of code that don't need to be indented or forget to indent lines that need to be indented. Seeing examples of these errors now will help you avoid them in the future and correct them when they do appear in your own programs.

INDENTATION ERROR

```
names = ['Shivam', 'Navneet', 'Anil']

for name in names:

print(f'{name} is champion')
```

LOGICAL ERROR

```
names = ['Shivam', 'Navneet', 'Anil']

for name in names:

    print(f'{name} is champion')

print('champion..champion..champion')
```

INDENTING UNNECESSARILY

```
names = ['Shivam', 'Navneet', 'Anil']

    for name in names:

        print(f'{name} is champion')

    print('champion..champion..champion')
```

FORGETTING THE COLON (so tough to find ☹)

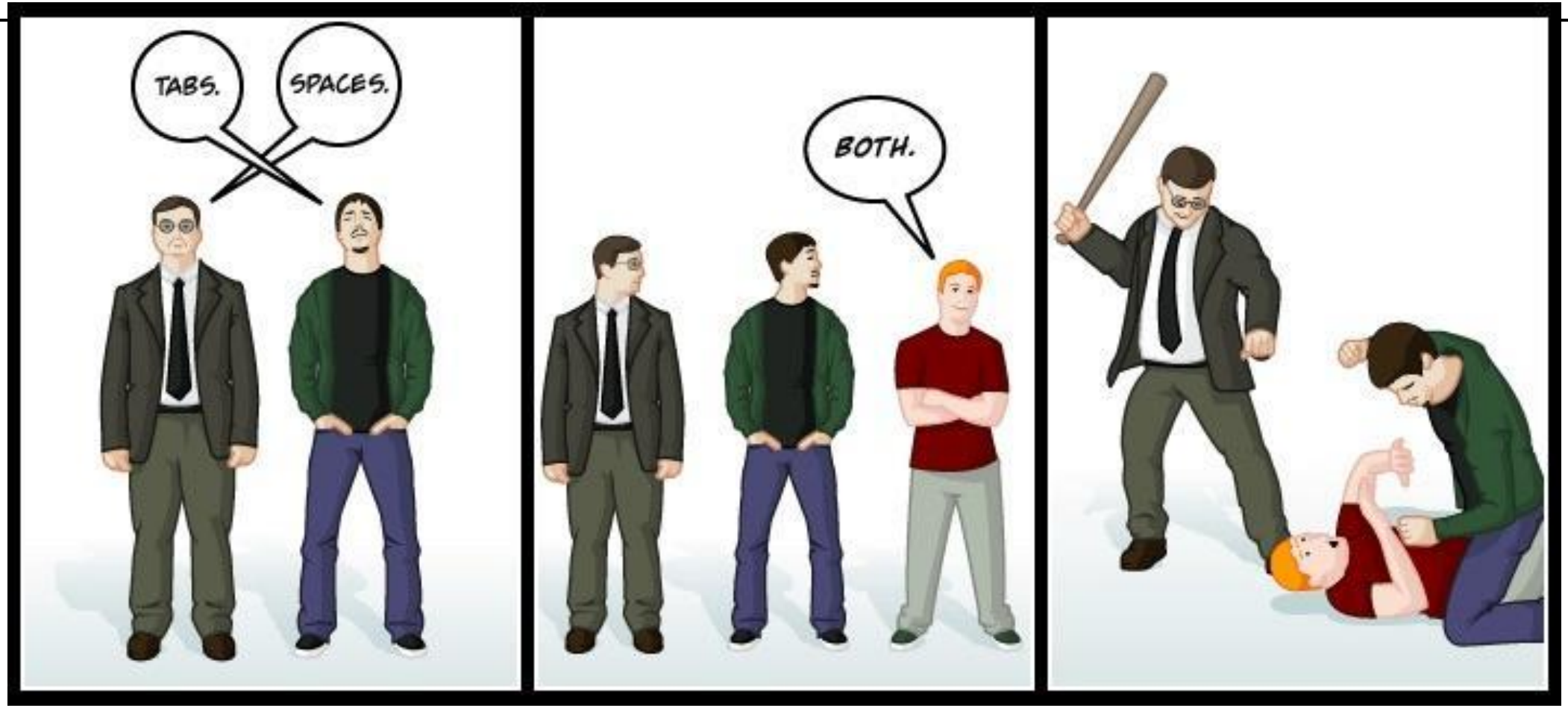
```
names = ['Shivam', 'Navneet', 'Anil']

for name in names

    print(f'{name} is champion')

    print('champion..champion..champion')
```

TABS VS SPACES FOR INDENTATION

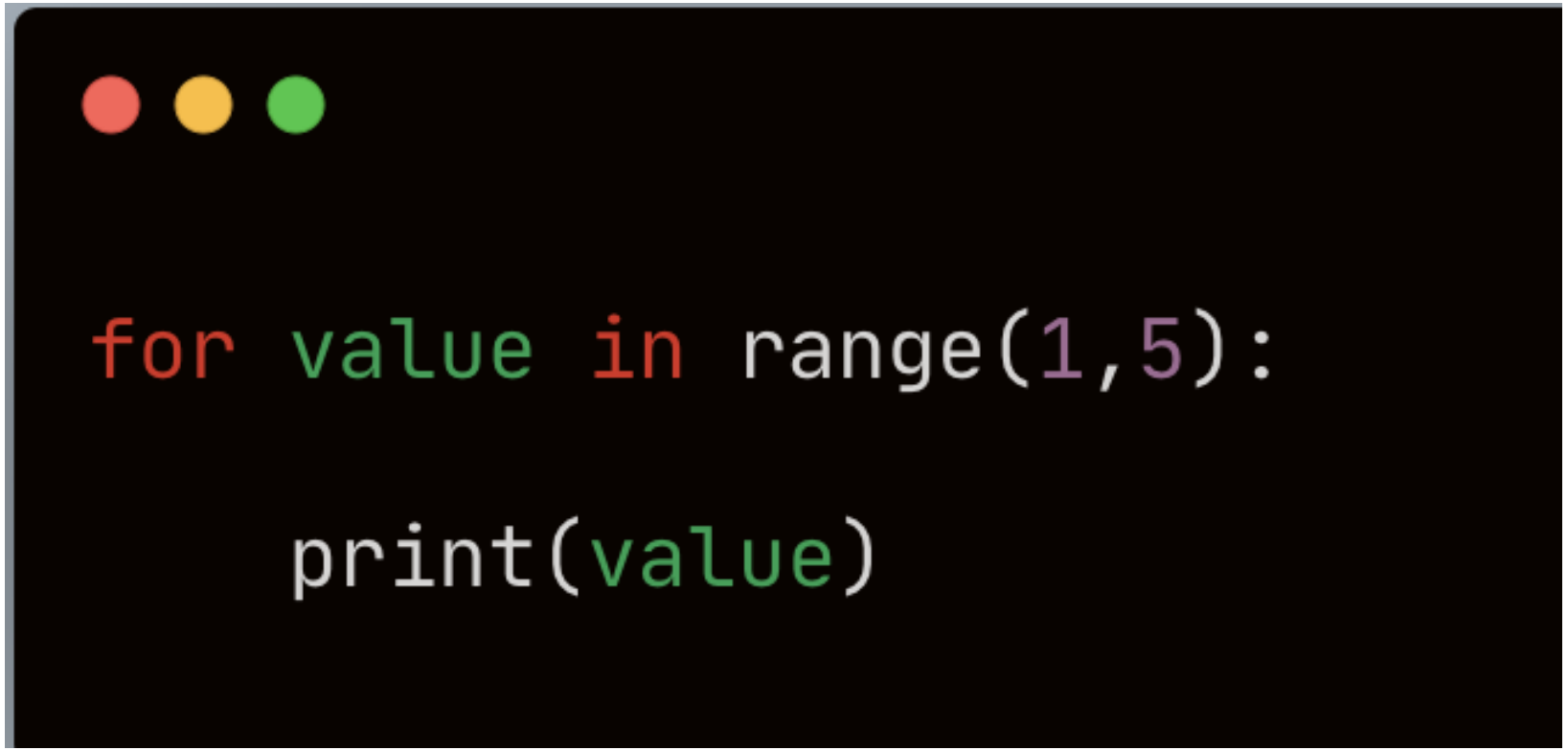


WHAT'S THE ACTUAL ISSUE ?

USING SPACES VS TABS

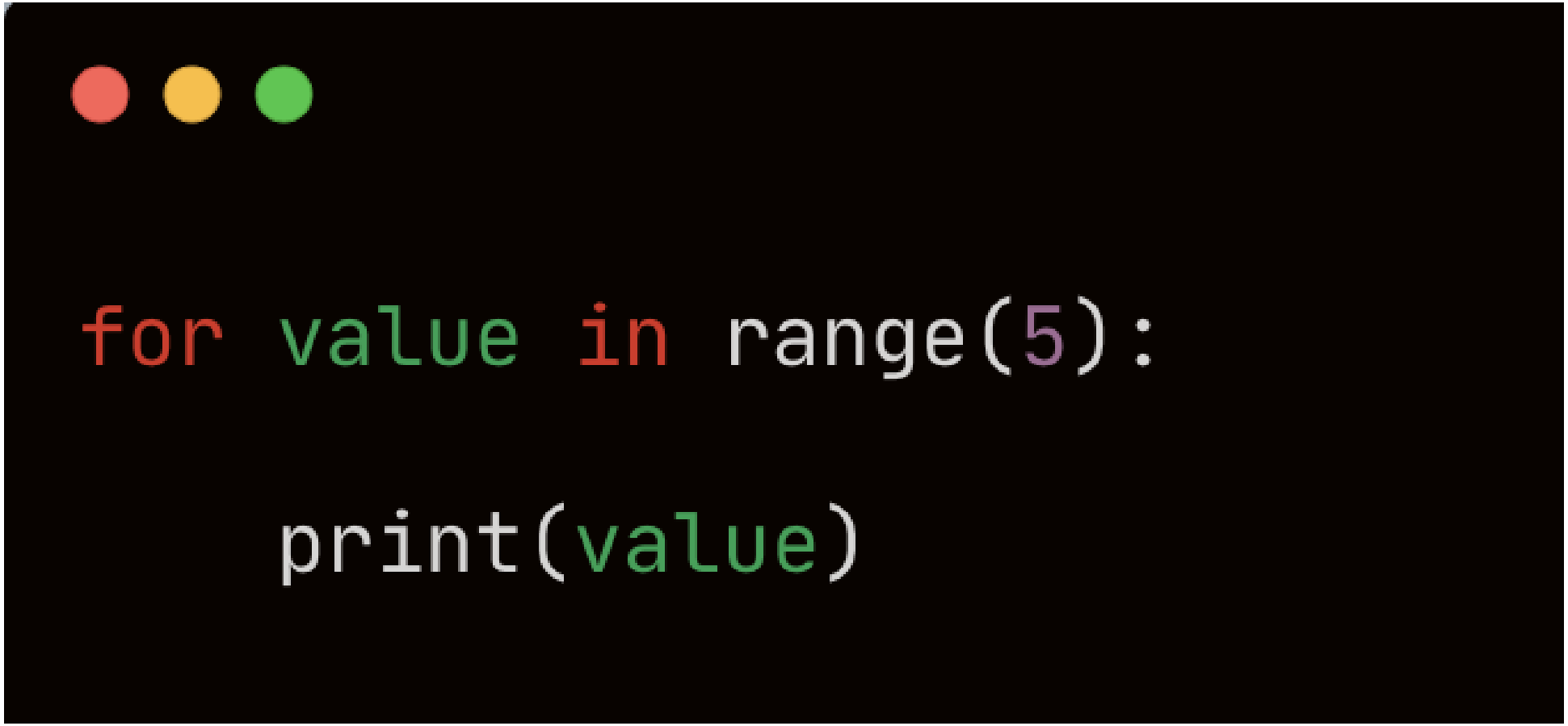
- Spaces - consistent on every text editor
- Tabs – not consistent on every text editor
  - Can be 4 spaces
  - Can be 8 spaces
- Mixing tabs and spaces causes error
- I prefer using 4 spaces for indentation
- Do not type spaces 4 times

RANGE FUNCTION

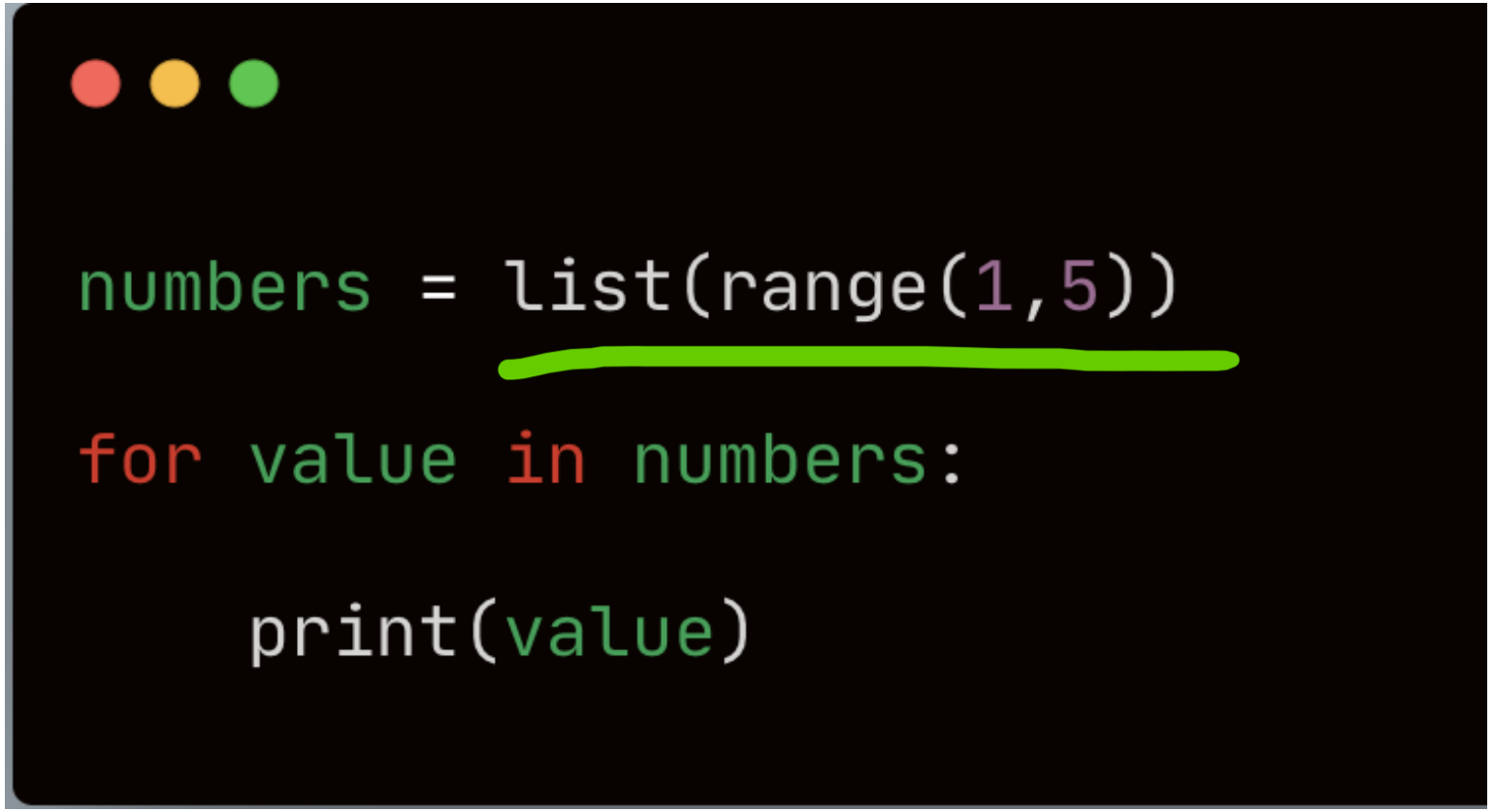


- Start to (end-1)
- Print 1 to 6
- Similar to other programming languages

RANGE FUNCTION



USING RANGE TO STORE A LIST OF NUMBERS

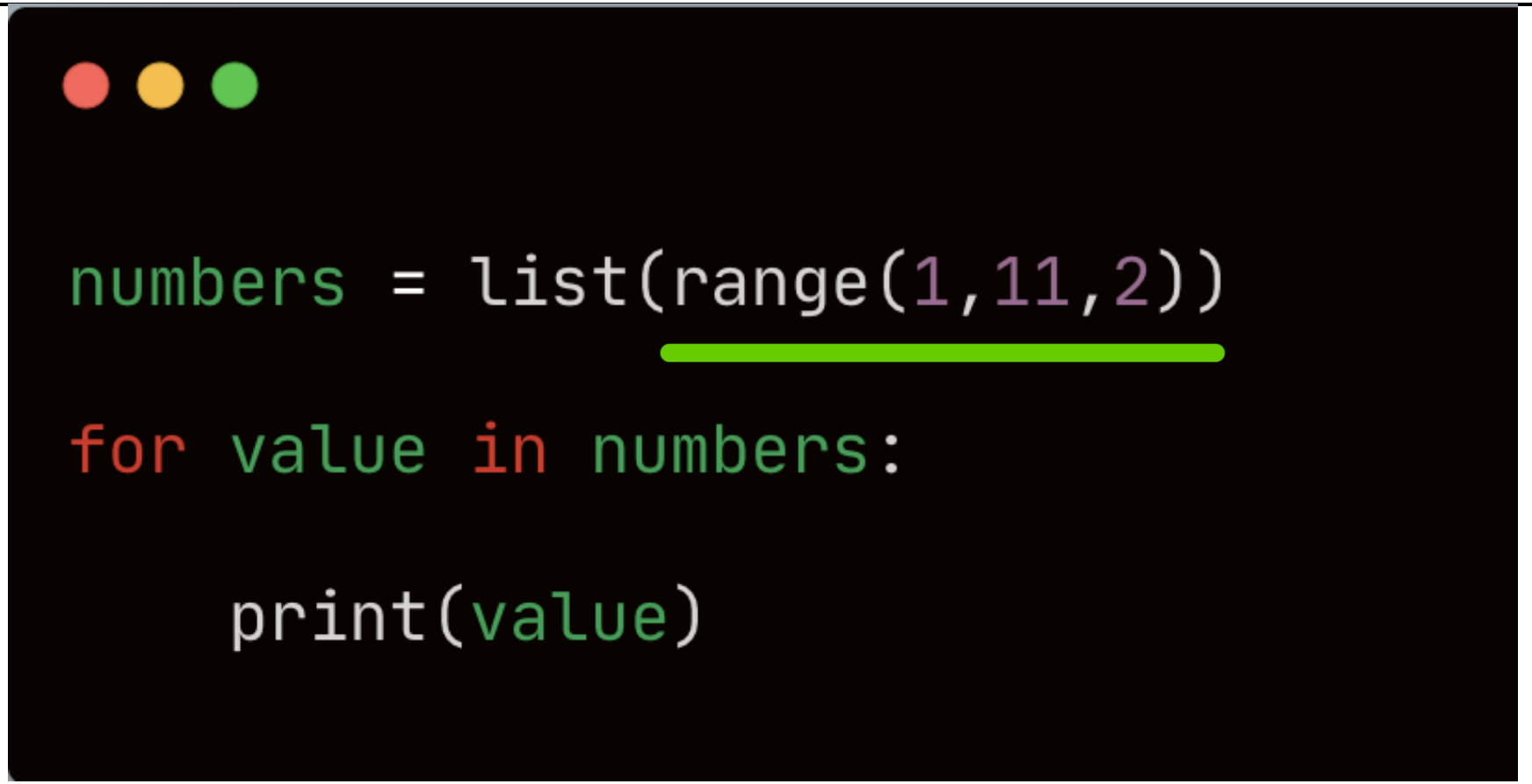


```
numbers = list(range(1,5))

for value in numbers:

    print(value)
```

SKIPPING NUMBERS WITHIN A RANGE



```
numbers = list(range(1,11,2))

for value in numbers:

    print(value)
```

PRINT MULTIPLICATION TABLE OF 5

Output – [5, 10, 15, ……., 50]

```
output = []
for value in range(1,11):
    num = 5 * value
    output.append(num)
print(output)
```

SIMPLE STATISTICS WITH A LIST OF NUMBERS

- Maximum of a list
- Minimum of a list
- Sum of a list

LIST COMPREHENSIONS

- A list comprehension combines the for loop and the creation of new elements into one line, and automatically appends each new element.
- Multiplication table in one line
- Not for beginner but you will see this in others code

```
output = [5*value for value in range(1,11)]

print(output)
```

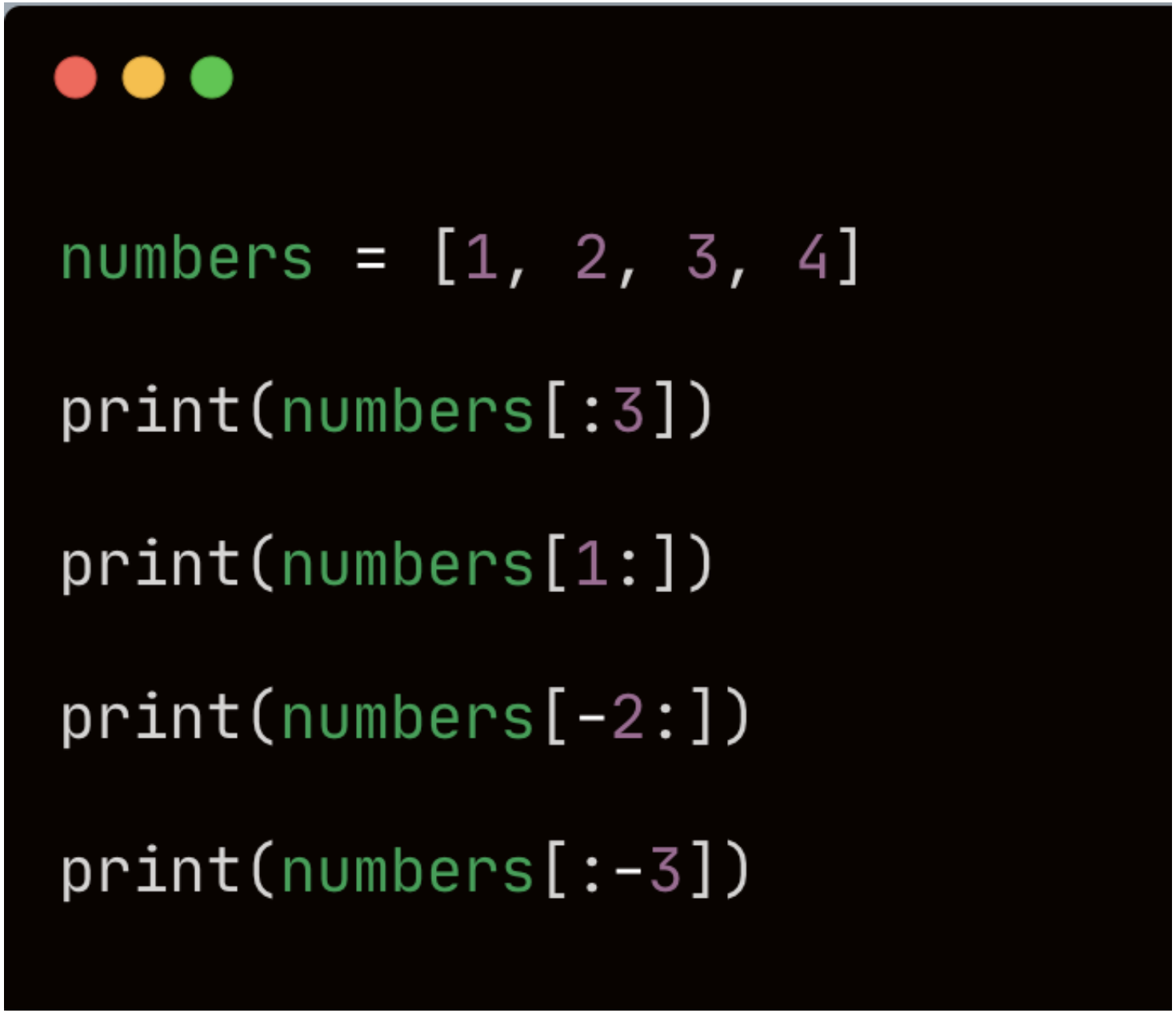
WORKING WITH A PART OF A LIST

- Can we access multiple list elements at one go ?
- Welcome to slice in python
- Slice of pizza ?
- Not present in all programming languages

SLICING A LIST

- To make a slice, you specify the index of the first and last elements you want to work with.
- Like range, the last element is not considered

```
numbers = [1, 2, 3, 4]
print(numbers[1:3])
print(numbers[0:4])
print(numbers[2:3])
print(numbers[2:2])
```



```
numbers = [1, 2, 3, 4]

print(numbers[:3])

print(numbers[1:])

print(numbers[-2:])

print(numbers[:-3])
```

LOOPING THROUGH A SLICE

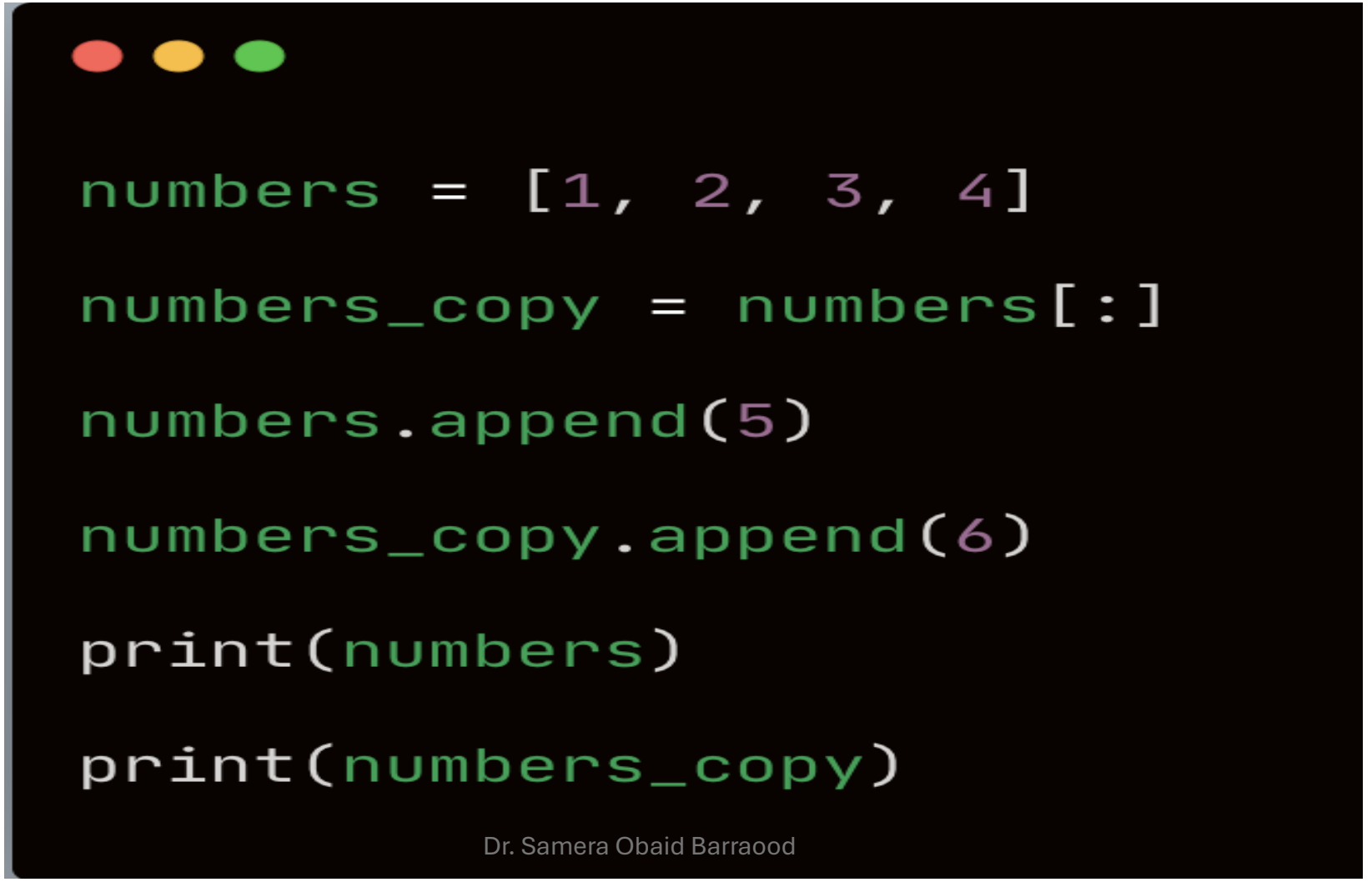
```python
numbers = [1, 2, 3, 4]

print("Printing the first thee numbers")

for num in numbers[:3]:

    print(num)
```

COPYING A LIST



```
numbers = [1, 2, 3, 4]

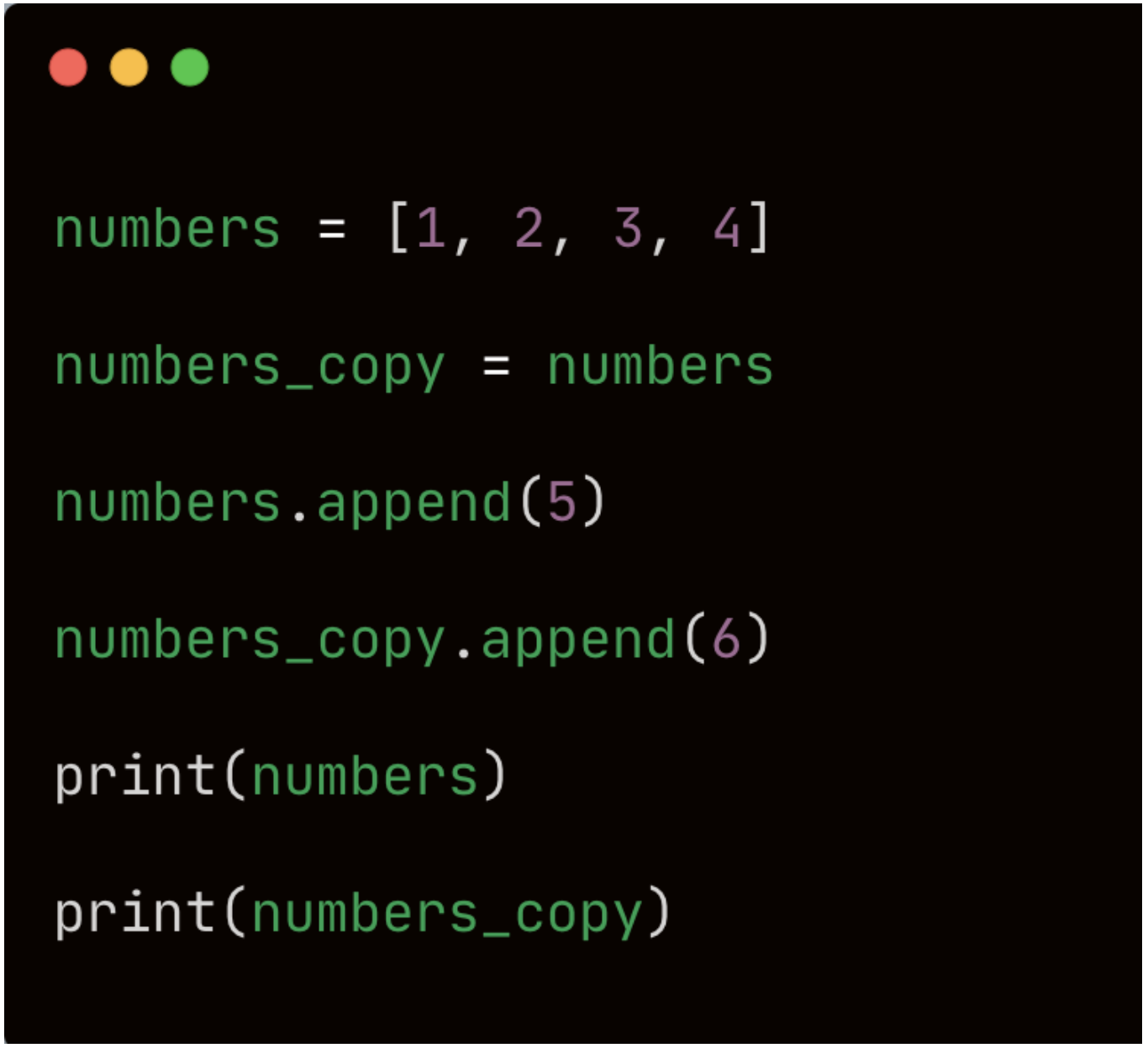
numbers_copy = numbers[:]

numbers.append(5)

numbers_copy.append(6)

print(numbers)

print(numbers_copy)
```



```
numbers = [1, 2, 3, 4]

numbers_copy = numbers

numbers.append(5)

numbers_copy.append(6)

print(numbers)

print(numbers_copy)
```

ASSIGNMENT

1. Print square of numbers from 1 to 10 using for loop
    1. Store it a list first and then print
    2. One line code
2. fruits = [‘banana’, ‘apple’, ‘watermelon’, ‘orange’]
    1. Print first 2 fruits (using slicing)
    2. Print last 2 fruits (suing slicing)
    3. Print fruits at odd positions (using for loop)
    4. Print fruits at even positions (using for loop)
3. Paste your code in the comment

Thank You

